FineSet â€” Complete Project Documentation

Document type: Technical project documentation
Audience: You, your team, and senior backend engineers
Format: Each section explains **what we did**, **why we did it**, and **how it works** with a simple example.

Table of contents

---

1. Product summary

What FineSet is

FineSet is a **multi-store jewelry retail SaaS**. One application serves many stores. Users work in three separate portals:

Portal	Who	Main work
Staff (`/staff/dashboard`)	Sales staff on the floor	Log visits, field sales, follow-up calls
Store (`/store/dashboard`)	Store manager	Monitor store data, staff, calls, analytics

Portal	Who	Main work
Admin (`/admin/dashboard`)	HQ / master admin	All stores, user invites, chain analytics

What problem it solves

Jewelry stores need to track:

- Every **customer visit** (did they buy? schemes pitched? follow-up date?)
- **Field sales** outside the store (events, door-to-door)
- **Follow-up calls** and outcomes
- **Performance metrics** per staff member (RSO)

FineSet replaces spreadsheets and disconnected tools with one system.

---

2. Technology choices

#	What we chose	Why (reason behind it)
1	**Next.js 16** (App Router)	One codebase for pages + API; deploy easily on Vercel; server components for fast dashboards
2	**TypeScript (strict)**	Same types in UI, API, and database layer â€” fewer bugs when fields change
3	**PostgreSQL on Supabase**	Reliable relational DB; managed hosting; built-in connection pooler
4	**Prisma**	Type-safe queries; migrations; seed scripts for demo data
5	**Supabase Auth**	We do not store user passwords ourselves; invite-by-email flow is built-in
6	**TanStack React Query**	Cache API data in the browser; refetch after mutations; works with SSR initial data
7	**Zod**	Validate API input and form input with the same rules
8	**Tailwind + shadcn/Radix**	Consistent UI quickly; accessible components

#	What we chose	Why (reason behind it)
9	**SSE (Server-Sent Events)**	Push â€œdata changedâ€ to open dashboards without full page reload
10	**Upstash Redis** (optional)	Rate limit login and writes in production
11	**AES-256-GCM encryption**	Protect customer name and phone in the database

---

3. Step-by-step: what we built and why

Below is the **order of thinking** we followed when building FineSet. Use this when explaining the project step by step.

Step 1 â€” Started with one Next.js application (not separate frontend + backend repos)

What we did:
Built UI and REST API inside the same Next.js project under `app/` and `lib/`.

Why:

- Small team: one deploy, one version, shared TypeScript types.
- Business logic lives in `lib/services/` â€” if we later extract a standalone API, we move that folder, not rewrite rules in the UI.
- Vercel runs Next.js natively.

Example:
`POST /api/visits` and the store visits page both call `createVisit()` in `lib/services/visits.ts`.

Files: `app/api/*`, `lib/services/*`

Step 2 â€” Designed the database around **Store** as the tenant boundary

What we did:
Every business table (Customer, Visit, FieldSale, Staff) has a `storeId` foreign key to `Store`.

Why:
Multi-tenant SaaS: Store Alpha must never see Store Betaâ€™s customers. Filtering by `storeId` is the main isolation rule.

Example:
When a store manager lists visits, the API sets `storeId = session.storeId` â€” they cannot request another storeâ€™s data.

Files: `prisma/schema.prisma` â€” model `Store`

Step 3 â€” Split **Staff** (operational) from **AppUser** (login)

What we did:

- `Staff` = employee on payroll (`employeeId` , name, store).
- `AppUser` = who can log in (`email` , `role` , link to `authId` from Supabase, optional `staffId`).

Why:

- Visits are attributed to **Staff** (`staffId`), not to a generic user id â€” metrics stay correct if login accounts change.
- A store manager may log in as `AppUser` without being a `Staff` row.
- Master admin has `AppUser` but no `Staff` row.

Example:
Staff member â€œEMP001â€ logs visits. Their `AppUser.staffId` points to that `Staff` row. Reports group by `Staff.employeeId` .

Files: `prisma/schema.prisma` â€” `Staff` , `AppUser`

Step 4 â€” Encrypted customer PII at the application layer

What we did:

- Before saving: encrypt `name` and `phone` with `ENCRYPTION_KEY` (AES-256-GCM).
- Store `phoneHash` (SHA-256 of digits only) for lookup and deduplication.
- When reading for authorized users: decrypt in `lib/services/pii.ts`.

Why:

- Phone numbers and names are sensitive (PII).
- Hash allows â€œfind customer by phoneâ€ without storing plaintext phone for search.
- Unique constraint `(phoneHash, storeId)` prevents duplicate customers per store.

Example:
Staff enters phone `9810001001`. DB stores encrypted blob + hash. Two visits with same phone upsert one `Customer` row.

Files: `lib/crypto/pii.ts`, `lib/services/pii.ts`, `lib/services/visits.ts`

Step 5 â€” Built a rich **Visit** model for jewelry retail workflows

What we did:
Visit table includes purchase status, intent tier, products, scheme pitching (GHS/GPP), enrollment outcome, follow-up flags, source channel, etc.

Why:
Store managers and HQ need analytics on schemes, conversion, and follow-ups â€” not just â€œa visit happened.â€

Example:
Staff marks visit as NOT_PURCHASED, HOT intent, schemes pitched GHS, follow-up in 3 days â† creates `Visit` + optional `FollowUp` row.

Files: `prisma/schema.prisma` â€” `Visit`, enums; `components/forms/VisitForm/`

Step 6 â€” Added **FieldSale** for off-store activity

What we did:
Separate model for door-to-door, events, corporate visits, etc., with its own enums (`FieldActivityType`, scheme outcomes).

Why:
Business process differs from in-store visits; managers want separate logs and reports.

Files: `prisma/schema.prisma` â€" `FieldSale` ; `app/api/field-sales/route.ts`

Step 7 â€" Linked **FollowUp** to Visit or FieldSale

What we did:
`FollowUp` has optional `visitId` OR `fieldSaleId` (one-to-one), plus `assignedStaffId` , status, call outcome fields.

Why:
Follow-up queue is the daily worklist for staff calls; must trace back to the original interaction.

Files: `prisma/schema.prisma` â€" `FollowUp` ; `lib/services/follow-ups.ts`

Step 8 â€" Added **StaffCallLog** and **PhoneRevealLog**

What we did:

- `StaffCallLog` : each call attempt on a visit (answered / not answered + feedback).
- `PhoneRevealLog` : audit when staff unmasks a phone number in the UI.

Why:
Managers need call discipline metrics; compliance needs traceability when PII is shown.

Files: `prisma/schema.prisma` ; `app/api/staff/calls/*`

Step 9 â€" Applied database migrations in order

What we did:

Migration	What changed
`20260526120000_init`	Full schema (stores, staff, visits, customers, â€¦)

Migration	What changed
`20260527120000_supabase_auth`	Added `AppUser`, `AuthAuditLog`; made old password columns optional
`20260527130000_remove_legacy_auth_columns`	Removed legacy app-managed password fields

Why:
We moved from early prototype auth (passwords in our DB) to **Supabase Auth** without losing business data.

Command: `npm run db:migrate`

Files: `prisma/migrations/*`

Step 10 â€” Integrated **Supabase Auth** for passwords and sessions

What we did:

- Supabase stores password hashes and session cookies.
- Our `AppUser` stores `authId` (Supabase user id), role, store, active flag.
- Login via server action `signInAction` â†' `supabase.auth.signInWithPassword`.

Why:

- Security: we do not implement password reset, bcrypt, or session crypto ourselves.
- Invite flow: `supabase.auth.admin.inviteUserByEmail` sends magic link; user sets password.

Example flow:

1. User enters email/password on `/login`.
2. Supabase validates â†' returns JWT in cookies.
3. `completeLoginForSupabaseUser` loads `AppUser`, checks `isActive`, syncs role into JWT metadata.

Files: `lib/auth/sign-in-action.ts`, `lib/auth/complete-login.ts`, `lib/supabase/server.ts`

Step 11 â€” Put roles in Postgres, cached in JWT app_metadata

What we did:
On login, sync `role`, `storeId`, `staffId` into Supabase `user.app_metadata`. `getAppSession()` reads metadata first; if incomplete, falls back to Prisma and syncs.

Why:

- Middleware (`proxy.ts`) can check role without a database query on every navigation.
- Faster dashboard load (target: under ~2 seconds).
- **Source of truth** remains `AppUser` in Postgres; metadata is a performance cache.

Files: `lib/auth/get-app-session.ts`, `lib/auth/session-from-metadata.ts`, `lib/auth/activate-profile.ts`

Step 12 â€” Protected portals with proxy.ts middleware

What we did:
`proxy.ts` runs on `/staff/dashboard/*`, `/store/dashboard/*`, `/admin/dashboard/*`:

1. Refresh Supabase session (cookies stay valid).
2. If not logged in â†' redirect to `/login`.
3. If logged in but wrong role for that URL â†' redirect with error.

Why:
Defense at the edge before any page renders â€" users cannot open another portalâ€™s URL by guessing.

Example:
Store manager JWT has `role: STORE_MANAGER`. Opening `/admin/dashboard` redirects to login with `wrong_portal`.

Files: `proxy.ts`, `lib/supabase/middleware.ts`

Step 13 â€” Used server-side layout guard as second layer

What we did:
Each portal layout calls `requirePortalSession("STAFF" | "STORE_MANAGER" | "MASTER_ADMIN")`.

Why:
Middleware can be bypassed in edge cases; server layout ensures API/RSC also see a valid session before rendering children.

Files: `app/(staff)/staff/dashboard/layout.tsx` , etc.; `lib/auth/require-portal-session.ts`

Step 14 â€” Implemented **invite** flow for new users

What we did:

- Admin: `POST /api/admin/users/invite`
- Store manager: `POST /api/store/users/invite`
- Creates `Staff` (if STAFF role), Supabase user via admin API, `AppUser` with `isActive: false` until first login.

Why:
Admins never see or set user passwords. User accepts email, sets password, profile activates.

Files: `lib/auth/invite-user.ts` , `app/auth/callback/route.ts`

Step 15 â€” Configured **two database URLs (pooler + direct)**

What we did:

- `DATABASE_URL` â†' Supabase pooler port `6543` with `pgbouncer=true` (runtime).
- `DIRECT_URL` â†' direct Postgres port `5432` (migrations only).

Why:
Serverless Next.js opens many short connections. Pooler prevents â€œtoo many connectionsâ€ errors on Supabase.

Files: `prisma/schema.prisma` datasource; `.env.local`

Step 16 â€” Created the services layer (`lib/services/*`)

What we did:
All Prisma writes/reads and business rules live here: `visits.ts` , `customers.ts` , `analytics.ts` , `follow-ups.ts` , etc.

Why:
Single place for rules. API routes stay thin. Server pages call the same functions (no duplicated SQL in components).

Rule:
API route = auth + Zod + call service + JSON response.

Files: `lib/services/*`

Step 17 â€” Built REST API routes under `app/api/`

What we did:
Standard HTTP handlers for visits, field-sales, staff, stores, analytics, follow-ups, calls, sync, invites.

Why:
Browser mutations and client-side filtering use `fetch()` to these endpoints. Mobile or external tools could use the same API later.

Example:
`GET /api/visits?page=1&pageSize=20` â†' `listVisits()` with store filter from session.

Files: `app/api/**/route.ts`

Step 18 â€” Validated every input with Zod

What we did:
Schemas in `lib/validations/*` (e.g. `createVisitSchema` , `getVisitsQuerySchema`). Used in API and forms via `@hookform/resolvers` .

Why:
Reject bad data at the boundary with clear 400 errors. Same rules on client and server â€" user sees consistent validation messages.

Files: `lib/validations/visit.schema.ts`, etc.

Step 19 â€” Implemented SSR + React Query hybrid for list pages

What we did:

1. **Server page** (`async` Server Component) calls `fetchInitialVisits()` in `lib/data/visits.ts`.
2. That calls `listVisits()` in services directly (no HTTP).
3. Passes `initialVisits` + `initialVisitsParams` to client component.
4. Client hook `useVisits()` uses that as `initialData` only if filters still match (`visitsParamsMatch`).
5. User changes page/search â†' hook fetches `GET /api/visits`.

Why:

Part	Benefit
SSR	First screen shows real data â€" no empty loading state
Same services	No duplicate business logic
React Query after	Smooth pagination without full page reload
Param matching	No stale SSR data when filters change

Files: `lib/data/visits.ts`, `hooks/useVisits.ts`, `lib/query/initial-data.ts`, `app/(store)/store/dashboard/visits/page.tsx`

Step 20 â€” Split Server Components vs Client Components

What we did:

- Default: Server Components (pages, layouts, static cards).
- `"use client"`: forms, tables with interaction, charts, `PortalShell`, providers.

Why:
Less JavaScript sent to browser; faster first paint; interactive parts isolated.

Files: `FE_ARCHITECTURE.md`, `components/forms/*`, `components/layout/PortalShell.tsx`

Step 21 â€” Centralized UI text in **content/en.ts**

What we did:
All labels, buttons, errors passed as `copy` props from `content/en.ts`.

Why:
One place to change wording; ready for future Hindi/other languages; consistent terms (GHS, GPP, RSO).

Files: `content/en.ts`

Step 22 â€” Built **VisitForm** and **FieldSalesForm** as client forms

What we did:
Large multi-section forms with react-hook-form, Zod resolver, submit â†' `useCreateVisit()` mutation â†' `POST /api/visits`.

Why:
Visit capture is the core staff workflow; must work on tablet/phone in store with validation and clear sections.

Files: `components/forms/VisitForm/`, `hooks/useVisits.ts`

Step 23 â€” Added **realtime sync** with SSE

What we did:

- After writes, `broadcastSyncEvent(storeId, entities)` in services.
- `GET /api/sync/events` streams version updates to browsers.
- `useRealtimeSync()` in portal layout invalidates React Query caches.

Why:
If manager and staff both have dashboards open, new visit appears on manager screen without manual refresh.

Limitation today: In-memory broadcaster works on **one server instance**. For multiple Vercel instances, next step is Redis pub/sub.

Files: `lib/sync/broadcaster.ts` , `app/api/sync/events/route.ts` , `hooks/useRealtimeSync.ts`

Step 24 â€” Added **rate limiting** (Upstash Redis)

What we did:
Limits on login, write APIs, and SSE connections when `UPSTASH_REDIS_*` env vars are set. Skipped gracefully in local dev without Redis.

Why:
Protect against brute-force login and accidental API abuse.

Files: `lib/rate-limit.ts`

Step 25 â€” Wrote **seed** and **bootstrap** scripts

What we did:

Command	Purpose
`npm run db:seed`	Demo stores, staff, customers, visits, field sales
`npm run auth:bootstrap`	Create production master admin from env vars
`npm run auth:bootstrap-dev`	Link dev logins to seeded staff/manager

Why:
New developers and demos work immediately without manual SQL.

Files: `prisma/seed.ts` , `scripts/bootstrap-admin.ts` , `scripts/bootstrap-dev-users.ts`

Step 26 â€” Built **analytics** services and dashboards

What we did:
API routes under `/api/analytics/admin` , `/api/analytics/store` , RSO performance endpoints.
Admin/store UI with charts (Recharts, lazy-loaded).

Why:
HQ compares stores; managers track staff performance and conversion â€" core business value of the SaaS.

Files: `lib/services/analytics.ts` , `lib/services/rso-performance.ts` , `components/admin/*`

Step 27 â€" Added **audit logging** for auth events

What we did:
`AuthAuditLog` table; `logAuthEvent()` on login success/failure, invites, etc.

Why:
Security review and troubleshooting (â€œwhy canâ€™t this user log in?â€).

Files: `lib/auth/audit.ts`

Step 28 â€" Set up **testing**

What we did:

Type	Tool	What it covers
Unit	Vitest	`lib/**` utilities, schemas, sync
Integration	Vitest	auth, security, sync against test DB patterns
E2E	Playwright	Golden paths per portal

Why:
Refactoring services and auth is safe when tests catch regressions.

Commands: `npm test` , `npm run test:integration` , `npm run test:e2e`

Step 29 â€" Deployed to **Vercel** with **Supabase** backend

What we did:
Production env vars on Vercel; Supabase Auth redirect URLs include production domain; pooler

URL for `DATABASE_URL`.

Why:
Git push â†' preview/production deploy; no server maintenance.

Files: `README.md`, `vercel.json` (if present)

Step 30 â€" Documented architecture for the team

What we did:
`ARCHITECTURE_GUIDE.md`, `FE_ARCHITECTURE.md`, `README.md`, and this document.

Why:
Onboarding and meetings with senior engineers â€" clear â€œwhat / why / how.â€□

---

4. Authentication journey

Diagram (login)

```
User â†' /login form
    â†' signInAction (server)
    â†' Supabase signInWithPassword
    â†' completeLoginForSupabaseUser
        â†' load AppUser (activate if invited)
        â†' sync app_metadata (role, storeId, staffId)
    â†' redirect to /staff|store|admin/dashboard
    â†' proxy.ts checks JWT role on each request
```

What we did vs what Supabase does

Task	Handled by
Password storage	Supabase
Session cookie / JWT	Supabase
Email invite link	Supabase Admin API

Task	Handled by
Role, store, staff assignment	Our `AppUser` table
â€œIs user allowed to use app?â€	`AppUser.isActive`
Portal access control	`proxy.ts` + API `requireRole`

Why we did not keep passwords in Postgres

Older migration made `Staff.passwordHash` optional, then removed legacy columns. Storing passwords in app DB duplicates Supabase, increases breach risk, and complicates invite/reset flows.

---

5. Database journey

Entity relationship (simple)

```
Store
â”œâ”€â”€ Staff (employeeId)
â”œâ”€â”€ AppUser (login)
â”œâ”€â”€ Customer (encrypted PII, phoneHash)
â”œâ”€â”€ Visit â†’ optional FollowUp, CallLogs
â””â”€â”€ FieldSale â†’ optional FollowUp
```

Important design decisions

Decision	Reason
`phoneHash` + `storeId` unique on Customer	Same phone at two stores = two customers; same store = one customer
Visit stores copy of customer name/phone	Historical record even if customer master updated later
Enums for status/outcomes	Valid data for analytics; no typos in free text
`onDelete: Restrict` on store	Prevent deleting store with existing visits

---

6. Backend journey

Request path (example: create visit)

```
 1. POST /api/visits + session cookie
2. getServerSession() â†’ STAFF + staffId + storeId
3. checkWriteRateLimit()
4. createVisitSchema.safeParse(body)
5. createVisit({ ...data, storeId, staffId })
6.   â†’ encrypt PII
7.   â†’ prisma.$transaction: customer upsert, visit create, followUp?
8.   â†’ broadcastSyncEvent
9. Return 201 JSON
```

Why transactions for visit create

If visit insert fails after customer upsert, we roll back everything â€” no orphan customer rows without visits.

---

7. Frontend journey

Three portals, shared patterns

Each portal has:

- `layout.tsx` â€” session guard + `PortalShell` + realtime sync
- Dashboard home â€” KPIs / quick actions
- List pages â€” SSR initial data + React Query
- Forms â€” client components for create flows

Staff typical day (UI flow)

1. Login â†’ staff dashboard
2. â€œLog visitâ€ â†’ VisitForm â†’ POST visit
3. â€œCallsâ€ â†’ follow-up queue â†’ log call outcome
4. â€œField salesâ€ â†’ FieldSalesForm â†’ POST field sale

Manager typical day

1. Login â†' store dashboard
2. Visits log â€" filter, search, export mindset
3. Calls â€" team queue
4. Analytics â€" period switcher, charts

---

8. Security and compliance

Measure	What we did	Why
PII encryption	AES-256-GCM in app	Protect data at rest in Postgres
Phone hashing	SHA-256	Search/dedup without decrypting
Masked display	Mask until â€œrevealâ€	Limit casual exposure on shared screens
Reveal audit	`PhoneRevealLog`	Know who viewed full phone
RBAC	Role on every API	Least privilege per portal
Rate limits	Upstash	Slow down abuse
Auth audit	`AuthAuditLog`	Investigate login issues
Service role key	Server only, never in browser	Admin invite API

---

9. Setup and deployment

Local setup (in order)

```
 npm install
cp .env.example .env.local   # fill DATABASE_URL, Supabase keys, ENCRYPTION_KEY
npm run db:migrate
npm run db:seed
npm run auth:bootstrap
npm run auth:bootstrap-dev
npm run dev
```

Why each command

Command	Why
`db:migrate`	Apply schema to Supabase
`db:seed`	Demo data for development
`auth:bootstrap`	Real master admin for production/testing
`auth:bootstrap-dev`	Password-known test accounts for seeded staff

Supabase dashboard settings

- Site URL: `http://localhost:3000` (and production URL)
- Redirect: `http://localhost:3000/auth/callback`

Production env (minimum)

`DATABASE_URL`, `DIRECT_URL`, `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `ENCRYPTION_KEY`, `NEXT_PUBLIC_APP_URL`

---

10. How to explain in a meeting

2-minute script

> We built FineSet as a Next.js SaaS for jewelry store chains. Supabase handles login; PostgreSQL holds stores, staff, encrypted customers, visits, and field sales. We use three portals â€" staff, store manager, and admin â€" with role checks in middleware and every API. All business logic is in a shared services layer so the API and server-rendered pages never duplicate rules. Staff log visits on the floor; managers see analytics and call queues. When data changes, we push SSE events so open dashboards refresh. We encrypt customer PII and audit phone reveals. New users are invited by email and activate on first login.

Suggested presentation order

1. Product + three portals (2 min)
2. Architecture diagram â€" browser â†' Next.js â†' services â†' Postgres + Supabase (3 min)

3. Auth split â€” Supabase vs AppUser (3 min)
4. Walk through one visit create â€” form â†' API â†' service â†' DB â†' SSE (5 min)
5. SSR + React Query pattern on visits list (3 min)
6. Security â€” PII, RBAC, rate limits (2 min)
7. Whatâ€™s next â€” Redis SSE, reporting scale (2 min)

---

11. Common questions and answers

Q: Why not a separate Node/Express backend?
A: Team size and speed. Logic is already isolated in `lib/services` for future extraction.

Q: How is multi-tenant isolation enforced?
A: `storeId` from session on every query; managers cannot override another store; staff writes forced to their `storeId` / `staffId`.

Q: Source of truth for role?
A: `AppUser.role` in Postgres. JWT metadata is synced on login for speed.

Q: Why encrypt in app vs database-native encryption?
A: Works with Prisma/Vercel uniformly; explicit control; tradeoff is key management via `ENCRYPTION_KEY`.

Q: Will SSE work on Vercel at scale?
A: Today in-memory per instance. Production scale needs Redis pub/sub between instances (Upstash already used for rate limits).

Q: Can mobile apps use this?
A: Yes â€” same `/api/*` endpoints with Supabase session cookies or token strategy if adapted later.

File index

Topic	Path
Database schema	`prisma/schema.prisma`
Seed data	`prisma/seed.ts`

Topic	Path
Visit service	`lib/services/visits.ts`
Visit API	`app/api/visits/route.ts`
Login	`lib/auth/sign-in-action.ts`
Session	`lib/auth/get-app-session.ts`
Middleware	`proxy.ts`
Invite	`lib/auth/invite-user.ts`
PII	`lib/crypto/pii.ts`
SSR data	`lib/data/visits.ts`
React Query hook	`hooks/useVisits.ts`
SSE	`app/api/sync/events/route.ts`
UI strings	`content/en.ts`
Setup	`README.md`

End of document â€" FineSet Complete Project Documentation